

1 Projekt i architektura systemu autoryzacji

Omawiane rozwiązanie zostało zaprojektowane jako modułowa biblioteka typu *Class Library* (.csproj), której głównym celem jest dostarczenie mechanizmu uwierzytelniania i autoryzacji dla aplikacji opartych o protokół HTTP. System charakteryzuje się architekturą pozwalającą na łatwą integrację z istniejącymi projektami poprzez wstrzykiwanie zależności oraz wykorzystanie atrybutów dekorujących punkty końcowe (ang. *endpoints*).

1.1 Koncepcja tokenu autoryzacyjnego

W przeciwieństwie do standardu JWT (JSON Web Token), zaprojektowany system wykorzystuje uproszczoną strukturę tokenu, zoptymalizowaną pod kątem specyficznych wymagań projektu. Token T jest ciągiem znaków składającym się z dwóch segmentów oddzielonych separatorem kropki. Każdy z segmentów jest zakodowany przy użyciu algorytmu Base64.

Strukturę tokenu można zapisać wzorem:

$$T = \text{Base64}(B) + "." + \text{Base64}(S)$$

gdzie:

- B - Ciało tokenu (Body), zawierające zserializowane dane użytkownika i metadane sesji.
- S - Suma kontrolna (Checksum), gwarantująca integralność danych.

Model danych zawarty w ciele tokenu (B) zdefiniowano następująco:

```
public sealed class Token
{
    public Guid Id { get; set; }
    public Guid UserId { get; set; }
    public object Data { get; set; }
    public string UserRoles { get; set; }
    public DateTime ExpirationDate { get; set; }
}
```

Zarówno tokeny dostępowe (ang. *Access Tokens*), jak i tokeny odświeżające (ang. *Refresh Tokens*) są przekazywane w nagłówkach zapytań HTTP, co zapewnia separację danych autoryzacyjnych od ciała żądania.

1.2 Warstwa danych i abstrakcja bazy

System został zaprojektowany z myślą o niezależności od konkretnego silnika bazodanowego. Domyślna implementacja wykorzystuje *Entity Framework Core* w połączeniu z Microsoft SQL Server, jednak architektura umożliwia:

1. Nadpisanie konfiguracji dla innych silników (np. MySQL, PostgreSQL).
2. Integrację z istniejącą bazą danych aplikacji nadrzędnej poprzez implementację odpowiednich interfejsów.

Schemat bazy danych opiera się na czterech kluczowych encjach: użytkownikach, rolach, powiązaniach użytkowników z rolami oraz tokenach odświeżających.

1.2.1 Definicje modeli encji

Poniżej przedstawiono strukturę klas reprezentujących tabele w bazie danych. Modele te stanowią fundament warstwa persystencji.

```
public sealed class RefreshToken
{
    public Guid Id { get; set; }
    public Guid UserId { get; set; }
    public DateTime ExpirationDate { get; set; }
}

public sealed class Role
{
    public Guid Id { get; set; }
```

```

        public int RoleNumber { get; set; }
        public string RoleCode { get; set; }
    }

    public class User
    {
        public Guid Id { get; set; }
        public string Username { get; set; }
        public string Email { get; set; }
        public string HashedPassword { get; set; }
    }

    public sealed class UserRole
    {
        public Guid Id { get; set; }
        public Guid UserId { get; set; }
        public Guid RoleId { get; set; }
    }

```

Bezpieczeństwo danych wrażliwych zapewnione jest poprzez mechanizmy kryptograficzne. Hasła użytkowników (*HashedPassword*) oraz sumy kontrolne tokenów podlegają procesowi szyfrowania przed zapisem lub transmisją.

2 Mechanizmy kryptograficzne i integralność danych

Bezpieczeństwo systemu autoryzacji opiera się na zapewnieniu integralności i niezaprzeczalności emitowanych tokenów. W tym celu zaimplementowano serwis *HashingService*, realizujący logikę podpisu cyfrowego oraz kodowania danych.

2.1 Algorytm podpisu cyfrowego

Do generowania sumy kontrolnej (podpisu) wykorzystano algorytm HMAC (ang. *Keyed-Hash Message Authentication Code*) w oparciu o funkcję skrótu SHA-256. Wybór ten podyktowany jest wysokim poziomem bezpieczeństwa oraz wydajnością tego rozwiązania w środowisku .NET.

Proces tworzenia tokenu T można przedstawić jako funkcję odwzorowującą obiekt danych D i klucz tajny K na łańcuch znaków:

$$\begin{aligned}
 P_{b64} &= \text{Base64Url}(\text{JSON}(D)) \\
 \sigma &= \text{Base64Url}(\text{HMAC}_{\text{SHA256}}(K, P_{b64})) \\
 T &= P_{b64} + "." + \sigma
 \end{aligned}$$

gdzie:

- P_{b64} – ładunek (payload) zakodowany w formacie Base64Url.
- σ – wygenerowany podpis cyfrowy.
- K – klucz symetryczny przechowywany w konfiguracji serwera, niedostępny dla klienta.

2.2 Implementacja serwisu haszującego

Klasa *HashingService*, implementująca interfejs *IHashingService*, jest odpowiedzialna za wszystkie operacje kryptograficzne związane z cyklem życia tokenu. Kluczowe dla implementacji było dostosowanie standardowego kodowania Base64 do wymogów przesyłania danych w protokole HTTP (tzw. Base64Url Encoding). Standardowe znaki + oraz / zostały zastąpione odpowiednio przez - i _, a znaki dopełnienia = zostały usunięte.

Poniższy listing prezentuje kod źródłowy serwisu:

```

using AJT.Contracts;
using AJT.Entities;
using AJT.Models;
using AJT.Options;
using Microsoft.Extensions.Options;

```

```

using Newtonsoft.Json;
using System.Security.Cryptography;
using System.Text;

namespace AJT.Services
{
    internal sealed class HashingService : IHashingService
    {
        private readonly IOptions<AJTOptions> _options;

        public HashingService(IOptions<AJTOptions> options)
        {
            _options = options;
        }

        public string Hash(Token token)
        {
            var tokenJson = JsonConvert.SerializeObject(token);
            var tokenJson64 = Base64UrlEncode(Encoding.UTF8.GetBytes(tokenJson));
            var signature = CreateSignature(tokenJson64, _options.Value.Secret);
            return string.Join('.', tokenJson64, signature);
        }

        public string Hash(RefreshToken refreshToken)
        {
            var tokenJson = JsonConvert.SerializeObject(refreshToken);
            var tokenJson64 = Base64UrlEncode(Encoding.UTF8.GetBytes(tokenJson));
            var signature = CreateSignature(tokenJson64, _options.Value.Secret);
            return string.Join('.', tokenJson64, signature);
        }

        public bool VerifiyHash(string token)
        {
            var parts = token.Split('.');
            if (parts.Length != 2)
                return false;

            string unsignedToken = parts[0];
            string signature = parts[1];
            string expectedSignature = CreateSignature(unsignedToken,
                                                        _options.Value.Secret);

            return signature == expectedSignature;
        }

        public string DecodePayload(string hashedToken)
        {
            var parts = hashedToken.Split('.');
            if (parts.Length != 2)
                throw new ArgumentException("Invalid AJT");

            var payload = parts[0];
            var bytes = Base64UrlDecode(payload);
            return Encoding.UTF8.GetString(bytes);
        }

        private static string CreateSignature(string unsignedToken, string secret)
        {
            var keyBytes = Encoding.UTF8.GetBytes(secret);
            var messageBytes = Encoding.UTF8.GetBytes(unsignedToken);

```

```

        using var hmac = new HMACSHA256(keyBytes);
        var hash = hmac.ComputeHash(messageBytes);
        return Base64UrlEncode(hash);
    }

    private static string Base64UrlEncode(byte[] input)
    {
        return Convert.ToBase64String(input)
            .TrimEnd('=')
            .Replace('+', '-')
            .Replace('/', '_');
    }

    private static byte[] Base64UrlDecode(string input)
    {
        string padded = input.Replace('-', '+').Replace('_', '/');
        switch (padded.Length % 4)
        {
            case 2: padded += "=="; break;
            case 3: padded += "="; break;
        }
        return Convert.FromBase64String(padded);
    }
}

```

2.3 Weryfikacja tokenu

Proces weryfikacji (`VerifyHash`) nie polega na odszyfrowaniu podpisu, lecz na jego ponownym wyliczeniu. System pobiera część tokenu zawierającą dane, generuje dla niej podpis przy użyciu tego samego sekretnego klucza K , a następnie porównuje wynik z podpisem przesłanym przez klienta. Zgodność obu ciągów znaków potwierdza, że dane nie zostały zmodyfikowane w trakcie transmisji oraz że token został wygenerowany przez zaufane źródło.

3 Mechanizm zarządzania rolami

Istotnym elementem systemu jest zoptymalizowany sposób przechowywania i weryfikacji uprawnień. Każda rola w systemie posiada unikalny identyfikator liczbowy $R_{id} \in \mathbb{N}$. Zbiór ról przypisanych do użytkownika nie jest przechowywany jako lista obiektów, lecz jako łańcuch znaków w formacie szesnastkowym (Hex), reprezentujący maskę bitową.

Mechanizm ten zakłada, że posiadanie roli o identyfikatorze n odpowiada ustawieniu n -tego bitu w ciągu binarnym na wartość 1. Otrzymana liczba jest następnie konwertowana na system szesnastkowy. Aby zminimalizować narzut danych przesyłanych w tokenie, zastosowano zmienną długość ciągu wynikowego – długość łańcucha Hex jest zależna od najwyższego identyfikatora roli posiadanej przez użytkownika.

$$UserRoles_{hex} = \text{Hex}\left(\sum_{i \in R_{user}} 2^i\right)$$

Taka konstrukcja pozwala na błyskawiczną weryfikację uprawnień poprzez operacje bitowe, bez konieczności iterowania po kolekcjach obiektów w trakcie przetwarzania każdego żądania HTTP.

3.1 Algorytmika obsługi ról bitowych

Kluczowym aspektem wydajnościowym systemu jest sposób reprezentacji ról. Zamiast przysyłać w tokenie pełne nazwy ról, system operuje na ciągu szesnastkowym (Hex), który jest reprezentacją tablicy bajtów.

Logika konwersji została zaimplementowana w serwisie `RoleService`. Metoda `DecodeRoles` przekształca ciąg Hex na tablicę bajtów, a następnie iteruje po poszczególnych bitach. Jeśli j -ty bit w i -tym bajcie jest ustawiony na 1, oznacza to, że użytkownik posiada rolę o numerze identyfikacyjnym obliczanym ze wzoru:

$$ID_{roli} = i \cdot 8 + j$$

Analogicznie, proces kodowania (`EncodeRoles`) polega na mapowaniu kodów ról na ich numery, a następnie ustawianiu odpowiednich bitów w tablicy bajtów przy użyciu logicznej sumy (OR) oraz przesunięcia bitowego. Poniższy fragment kodu prezentuje implementację metod kodujących i dekodujących role:

```
public async Task<List<string>> DecodeRoles(string userRolesString)
{
    using var scope = _scopeFactory.CreateScope();
    var roleRepo = scope.ServiceProvider.GetRequiredService<IRoleRepo>();
    var dbRoles = await roleRepo.GetAllRoles();

    int byteCount = userRolesString.Length / 2;
    byte[] bytes = new byte[byteCount];
    for (int i = 0; i < byteCount; i++)
        bytes[i] = Convert.ToByte(userRolesString.Substring(i * 2, 2), 16);

    var result = new List<string>();

    for (int byteIndex = 0; byteIndex < bytes.Length; byteIndex++)
    {
        for (int bit = 0; bit < 8; bit++)
        {
            int bitIndex = byteIndex * 8 + bit;
            bool isSet = (bytes[byteIndex] & (1 << bit)) != 0;
            if (isSet)
            {
                var role = dbRoles.FirstOrDefault(r =>
                    r.RoleNumber == bitIndex)?.RoleCode;
                if (role != null)
                    result.Add(role);
            }
        }
    }
    return result;
}

public async Task<string> EncodeRoles(List<string> roles)
{
    using var scope = _scopeFactory.CreateScope();
    var roleRepo = scope.ServiceProvider.GetRequiredService<IRoleRepo>();
    var dbRoles = await roleRepo.GetAllRoles();
    var userRoles = new Dictionary<int, string>();

    int maxBit = 0;
    foreach (var role in roles)
    {
        var roleNumber = dbRoles.FirstOrDefault(x =>
            x.RoleCode == role)?.RoleNumber ?? 0;
        userRoles.Add(roleNumber, role);
        if (maxBit < roleNumber)
            maxBit = roleNumber;
    }

    int byteCount = (maxBit / 8) + 1;
    byte[] bytes = new byte[byteCount];

    foreach (var kv in userRoles)
    {
        int bitIndex = kv.Key;
        int byteIndex = bitIndex / 8;
        int bitInByte = bitIndex % 8;
        bytes[byteIndex] |= (byte)(1 << bitInByte);
    }
}
```

```

    }

    return BitConverter.ToString(bytes).Replace("-", "");
}

```

Takie podejście pozwala na znaczną redukcję rozmiaru tokenu, co jest istotne w kontekście przesyłania danych w nagłówkach HTTP przy każdym zapytaniu.

4 Logika biznesowa procesu uwierzytelniania

Sercem modułu odpowiedzialnym za zarządzanie tożsamością użytkowników jest serwis `LoginService`. Klasa ta agreguje logikę rejestracji, logowania oraz odświeżania sesji, wykorzystując wcześniej zdefiniowane komponenty kryptograficzne oraz warstwę dostępu do danych.

4.1 Proces logowania i generowania tokenów

Proces uwierzytelniania (metoda `Login`) przebiega według ściśle określonego schematu:

1. Pobranie danych uwierzytelniających (login i hasło).
2. Wyznaczenie skrótu hasła przy użyciu wstrzykniętego serwisu `IPasswordHasher`.
3. Weryfikacja tożsamości w bazie danych poprzez porównanie skrótów.
4. W przypadku sukcesu – wygenerowanie pary tokenów: dostępowego (Token) oraz odświeżającego (Refresh Token).

Kluczowym elementem jest metoda `CreateCombinedTokenForUser`, która orkiestruje budowę finalnego obiektu. W tym kroku następuje:

- Pobranie ról użytkownika i ich kompresja do formatu Hex (zgodnie z opisanym wcześniej algorytmem).
- Pobranie dodatkowych, niestandardowych danych (ang. *custom claims*) zdefiniowanych przez programistę aplikacji nadrzędnej.
- Podpisanie obu tokenów i zwrócenie ich w obiekcie `CombinedToken`.

Poniższy listing przedstawia implementację serwisu logowania:

```

using AJT.Contracts;
using AJT.Entities;
using AJT.Models;
using AJT.Options;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Options;
using Newtonsoft.Json;

namespace AJT.Services
{
    internal sealed class LoginService : ILoginService
    {
        private readonly IServiceScopeFactory _scopeFactory;
        private readonly IOptions<AJTOptions> _options;
        private readonly IPasswordHasher _passwordHasher;
        private readonly IRoleService _roleService;
        private readonly IHashingService _hashingService;
        private readonly ITokenDataService _tokenDataService;

        public LoginService(IServiceScopeFactory scopeFactory,
                            IOptions<AJTOptions> options,
                            IPasswordHasher passwordHasher,
                            IRoleService roleService,
                            IHashingService hashingService,
                            ITokenDataService tokenDataService)

```

```

{
    _scopeFactory = scopeFactory;
    _options = options;
    _passwordHasher = passwordHasher;
    _roleService = roleService;
    _hashingService = hashingService;
    _tokenDataService = tokenDataService;
}

public async Task<CombinedToken?> Login(string login, string password)
{
    var hashedPassword = _passwordHasher.HashPassword(password);

    using var scope = _scopeFactory.CreateScope();
    var userRepo = scope.ServiceProvider.GetRequiredService<IUserRepo>();
    var user = await userRepo.GetUserByLogin(login);

    if (user is null || user.HashedPassword != hashedPassword)
        return null;

    return await CreateCombinedTokenForUser(scope, user);
}

public async Task<CombinedToken?> Refresh(string refreshTokenString)
{
    if (!_hashingService.VerifyHash(refreshTokenString))
        return null;

    RefreshToken refreshToken = null;
    try
    {
        var json = _hashingService.DecodePayload(refreshTokenString);
        refreshToken = JsonConvert.DeserializeObject<RefreshToken>(json);
        if (refreshToken is null)
            return null;
    }
    catch
    {
        return null;
    }

    if (refreshToken.ExpirationDate < DateTime.Now)
        return null;

    using var scope = _scopeFactory.CreateScope();
    var userRepo = scope.ServiceProvider.GetRequiredService<IUserRepo>();
    var user = await userRepo.GetUserById(refreshToken.UserId);

    if (user is null)
        return null;

    return await CreateCombinedTokenForUser(scope, user);
}

private async Task<CombinedToken?> CreateCombinedTokenForUser(
    IServiceScope scope, User user)
{
    var userRoleRepo = scope.ServiceProvider
        .GetRequiredService<IUserRoleRepo>();
    var roles = await userRoleRepo.GetUserRoles(user);

```

```

var token = new Token
{
    Id = Guid.NewGuid(),
    UserId = user.Id,
    Data = await _tokenDataService.GetCustomTokenData(user.Id),
    ExpirationDate = DateTime.Now.Add(_options.Value.TokenExpirationTime)
};

if (roles is not null)
    token.UserRoles = await _roleService.EncodeRoles(
        roles.Select(x => x.RoleCode).ToList());

var refreshTokenRepo = scope.ServiceProvider
    .GetRequiredService<IRefreshTokenRepo>();

var refreshToken = new RefreshToken
{
    UserId = user.Id,
    ExpirationDate = DateTime.Now.Add(
        _options.Value.RefreshTokenExpirationTime)
};
await refreshTokenRepo.AddRefreshToken(refreshToken);

return new CombinedToken
{
    Token = _hashingService.Hash(token),
    RefreshToken = _hashingService.Hash(refreshToken)
};
}
}
}

```

5 Automatyzacja zarządzania uprawnieniami

W celu zapewnienia spójności między kodem aplikacji a stanem bazy danych, system został wyposażony w moduł automatycznej synchronizacji ról. Mechanizm ten, zaimplementowany w klasie `RoleBootstrapper`, wykorzystuje interfejs `IHostedService`, co pozwala na uruchomienie procedury inicjalizacyjnej w tle, bezpośrednio po starcie aplikacji (w fazie *startup*).

5.1 Algorytm synchronizacji i detekcji ról

Serwis `RoleBootstrapper` realizuje dwa tryby pracy, zależne od konfiguracji:

1. **Tryb statyczny** – role są pobierane z predefiniowanej listy w pliku konfiguracyjnym lub kodzie startowym.
2. **Tryb automatycznej detekcji** – system wykorzystuje mechanizm refleksji (*System.Reflection*) do analizy zestawu instrukcji (*Assembly*).

W trybie automatycznym algorytm przeszukuje wszystkie klasy i metody w poszukiwaniu atrybutu `AllowRoleAttribute`. Unikalne nazwy ról zebrane z atrybutów tworzą zbiór definicji, który musi zostać odzwierciedlony w bazie danych. Jest to kluczowe dla poprawnego działania mechanizmu bitowego mapowania ról, który wymaga, aby każda rola posiadała unikalny, sekwencyjny identyfikator liczbowy.

5.2 Procedura aktualizacji bazy danych

W przypadku wykrycia rozbieżności między definicjami w kodzie a stanem bazy danych (np. dodanie nowej roli, zmiana kolejności wpływająca na ID), uruchamiana jest procedura `ChangeRolesAsync`. Ze względu na to, że zmiana identyfikatora liczbowego roli (jej pozycji w masce bitowej) invaliduje dotychczasowe ciągi Hex przypisane użytkownikom, proces ten jest złożony i składa się z następujących kroków:

1. Pobranie aktualnych ról oraz przypisanie użytkowników do pamięci operacyjnej (backup logiczny).
2. Usunięcie wszystkich definicji ról z bazy danych.

3. Wstawienie nowych definicji ról z poprawnymi, sekwencyjnymi identyfikatorami.
4. Odtworzenie powiązań użytkowników na podstawie nazw ról (mapowanie nazwa → nowe ID).

Poniższy listing prezentuje implementację serwisu synchronizującego:

```
using AJT.Attributes;
using AJT.Contracts;
using AJT.Options;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Options;
using System.Reflection;

namespace AJT.Services
{
    internal class RoleBootstrapper : IHostedService
    {
        private readonly IServiceScopeFactory _scopeFactory;
        private readonly IOptions<AJTOptions> _options;

        public RoleBootstrapper(IServiceScopeFactory scopeFactory,
                                IOptions<AJTOptions> options)
        {
            _scopeFactory = scopeFactory;
            _options = options;
        }

        public async Task StartAsync(CancellationToken cancellationToken)
        {
            using var scope = _scopeFactory.CreateScope();
            var roleRepo = scope.ServiceProvider.GetRequiredService<IRoleRepo>();

            var existingRoles = await roleRepo.GetAllRoles();
            var configRoles = new List<(int Id, string Code)>();

            var list = _options.Value.DetectRolesFromAssembly
                ? GetDefinedRoles()
                : _options.Value.Roles;

            if (list is null || list.Count == 0)
                return;

            for (int i = 0; i < list.Count; i++)
                configRoles.Add(new(i, list[i]));

            foreach (var (number, code) in configRoles)
            {
                var role = existingRoles.FirstOrDefault(x => x.RoleCode == code);

                if (role is null || role.RoleNumber != number)
                {
                    await ChangeRolesAsync(configRoles, cancellationToken);
                    return;
                }
            }
        }

        private async Task ChangeRolesAsync(List<(int number, string Code)> roles,
                                             CancellationToken cancellationToken)
        {
            // Kopia zapasowa przypisać użytkowników
        }
    }
}
```

```

using var scope = _scopeFactory.CreateScope();
var roleRepo = scope.ServiceProvider.GetRequiredService<IRoleRepo>();
var userRoleRepo = scope.ServiceProvider
    .GetRequiredService<IUserRoleRepo>();

var currnetRoles = await roleRepo.GetAllRoles();
var userRoles = await userRoleRepo.GetAllUserRoles();
var userRoleNames = userRoles.Select(x => new
{
    x.UserId,
    RoleCode = currnetRoles.Find(role => role.Id == x.RoleId)?.RoleCode
}).ToList();

// Reset tabeli ról
await roleRepo.RemoveAllRoles();

// Wstawienie nowych definicji
foreach (var (roleNumber, roleCode) in roles)
    await roleRepo.AddRole(new Entities.Role()
    {
        RoleNumber = roleNumber,
        RoleCode = roleCode
    });

// Przywrócenie przypisań
var updatedRoles = await roleRepo.GetAllRoles();

foreach (var userRole in userRoleNames)
{
    var role = updatedRoles.FirstOrDefault(x =>
        x.RoleCode == userRole.RoleCode);
    if (role is null)
        continue;

    await userRoleRepo.AddUserRole(new Entities.User()
        { Id = userRole.UserId }, role);
}
}

public Task StopAsync(CancellationTokens cancellationTokens)
{
    return Task.CompletedTask;
}

private List<string> GetDefinedRoles()
{
    var assembly = Assembly.GetExecutingAssembly();

    // Ekstrakcja ról z klas
    var classRoles = assembly.GetTypes()
        .SelectMany(t => t.GetCustomAttributes<AllowRoleAttribute>(true))
        .SelectMany(a => a.Roles);

    // Ekstrakcja ról z metod
    var methodRoles = assembly.GetTypes()
        .SelectMany(t => t.GetMethods())
        .SelectMany(m => m.GetCustomAttributes<AllowRoleAttribute>(true))
        .SelectMany(a => a.Roles);

    return classRoles
        .Concat(methodRoles)

```

```

        .Distinct()
        .ToList();
    }
}

```

6 Dynamiczne rozszerzanie ładunku tokenu

Jednym z założeń projektowych była możliwość przekazywania w tokenie dodatkowych informacji, specyficznych dla domeny aplikacji korzystającej z biblioteki (np. identyfikator oddziału firmy, poziom subskrypcji). Aby uniknąć sztywnego wiązania struktury tokenu z modelem biznesowym, zastosowano mechanizm delegatów.

6.1 Implementacja serwisu TokenDataService

Klasa `TokenDataService` pełni rolę pośrednika, który umożliwia wstrzyknięcie dowolnego obiektu do pola `Data` w tokenie. Mechanizm ten opiera się na delegacie `Func`, który przyjmuje identyfikator użytkownika oraz dostawcę usług (`IServiceProvider`).

Dostęp do `IServiceProvider` wewnątrz delegata jest kluczowy – pozwala on programiście na rozwiązanie dowolnej zależności (np. repozytorium użytkowników, zewnętrznego API) w celu pobrania wymaganych danych w momencie generowania tokenu.

Implementację serwisu przedstawiono poniżej:

```

using AJT.Contracts;

namespace AJT.Services
{
    internal sealed class TokenDataService : ITokenDataService
    {
        private Func<Guid, IServiceProvider, Task<object>>? _func;
        private readonly IServiceProvider _serviceProvider;

        public static Func<Guid, IServiceProvider, Task<object>>? InitFunc { get; set; }

        public TokenDataService(IServiceProvider serviceProvider)
        {
            _serviceProvider = serviceProvider;
            _func = InitFunc;
        }

        public async Task<object?> GetCustomTokenData(Guid userId)
        {
            if (_func is null)
                return null;

            return await _func(userId, _serviceProvider);
        }

        public void RegisterAction(Func<Guid, IServiceProvider, Task<object>> func)
        {
            _func = func;
        }
    }
}

```

7 Implementacja warstwy autoryzacji w potoku HTTP

Weryfikacja dostępu do poszczególnych zasobów (Endpointów) realizowana jest poprzez atrybuty (filtry) w architekturze ASP.NET Core. System rozróżnia dwa poziomy dostępu:

1. Wymagane posiadanie ważnego tokenu.

2. Wymagane posiadanie ważnego tokenu oraz określonej roli (lub ról).

W celu integracji z potokiem przetwarzania żądań (ang. *request pipeline*) frameworka ASP.NET Core, wykorzystano mechanizm filtrów. Ze względu na konieczność wstrzykiwania zależności (takich jak serwisy haszujące czy repozytoria ról) do atrybutów, zastosowano klasę `TypeFilterAttribute`. Pozwala ona na instancjonowanie filtrów przez kontener IoC (Inversion of Control), co nie jest możliwe w przypadku standardowych atrybutów dziedziczących bezpośrednio po `Attribute`.

Główna logika decyzyjna została zawarta w klasie `AllowRoleFilter`, implementującej interfejs `IAsyncAuthorizationFilter`. Proces autoryzacji przebiega według następującego algorytmu:

1. Pobranie nagłówka `Authorization`.
2. Weryfikacja obecności i formatu schematu `Bearer`.
3. Walidacja sumy kontrolnej (podpisu) tokenu.
4. Deserializacja ładunku (ang. *payload*) oraz sprawdzenie daty wygaśnięcia.
5. W przypadku atrybutu wymagającego ról – dekodowanie uprawnień użytkownika i konfrontacja z wymaganymi rolami.

Poniższy listing przedstawia implementację filtra weryfikującego role:

```
using AJT.Contracts;
using AJT.Exceptions;
using AJT.Models;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;
using Newtonsoft.Json;

namespace AJT.Attributes
{
    internal sealed class AllowRoleFilter : Attribute, IAsyncAuthorizationFilter
    {
        private readonly string[] _roles;
        private readonly IHashingService _hashingService;
        private readonly IRoleService _roleService;

        public AllowRoleFilter(string[] roles, IHashingService hashingService,
                               IRoleService roleService)
        {
            _roles = roles;
            _hashingService = hashingService;
            _roleService = roleService;
        }

        public async Task OnAuthorizationAsync(AuthorizationFilterContext context)
        {
            var authHeader = context.HttpContext.Request.Headers["Authorization"]
                               .ToString();

            if (string.IsNullOrEmpty(authHeader)
                || !authHeader.StartsWith("Bearer "))
            {
                context.HttpContext.Response.StatusCode =
                    StatusCodes.Status401Unauthorized;
                await context.HttpContext.Response.WriteAsync(
                    JsonConvert.SerializeObject(new { message = "Unauthorized" }));
                return;
            }

            var token = authHeader["Bearer ".Length..].Trim();
            if (!_hashingService.VerifyHash(token))
```

```

{
    context.HttpContext.Response.StatusCode =
        StatusCodes.Status401Unauthorized;
    await context.HttpContext.Response.WriteAsync(
        JsonConvert.SerializeObject(new { message = "Unauthorized" }));
    return;
}

try
{
    var payloadJson = _hashingService.DecodePayload(token);
    var tokenObject = JsonConvert.DeserializeObject<Token>(payloadJson);

    if (tokenObject is null || string.IsNullOrEmpty(tokenObject.UserRoles))
        throw new UnauthorizedException();

    var data = tokenObject.Data;
    if (data is not null)
        context.HttpContext.Items["AJT"] = data;

    if (tokenObject.ExpirationDate < DateTime.Now)
    {
        throw new UnauthorizedException();
    }

    var userRoles = await _roleService.DecodeRoles(tokenObject.UserRoles);
    foreach (var role in _roles)
        if (userRoles.Contains(role))
            return;

    throw new ForbidException();
}
catch (ForbidException)
{
    context.HttpContext.Response.StatusCode = StatusCodes.Status403Forbidden;
    await context.HttpContext.Response.WriteAsync(
        JsonConvert.SerializeObject(new { message = "Forbidden" }));
    return;
}
catch (Exception)
{
    context.HttpContext.Response.StatusCode =
        StatusCodes.Status401Unauthorized;
    await context.HttpContext.Response.WriteAsync(
        JsonConvert.SerializeObject(new { message = "Unauthorized" }));
    return;
}
}
}

```

Dla punktów końcowych niewymagających konkretnej roli, a jedynie faktu uwierzytelnienia, przygotowano lżejszy filtr `RequireAuthenticationFilter`, który pomija etap dekodowania ról, kończąc walidację na sprawdzeniu poprawności kryptograficznej i ważności czasowej tokenu.

8 Konfiguracja i integracja systemu

Aby umożliwić elastyczną integrację biblioteki z różnymi projektami, zastosowano wzorzec *Builder* oraz mechanizm *Extension Methods* dla kontenera wstrzykiwania zależności (DI) platformy .NET.

Klasa `AJTConfigurator` pozwala na łańcuchowe definiowanie parametrów systemu, takich jak:

- Implementacja serwisu haszującego hasła.
- Automatyczna detekcja ról.
- Konfiguracja migracji bazy danych.
- Definicja funkcji wstrzykującej dodatkowe dane do tokenu.

Metoda rozszerzająca `UseAJT` jest punktem wejścia dla biblioteki. Rejestruje ona w kontenerze IoC wszystkie niezbędne serwisy wewnętrzne (repozytoria, serwisy domeny) oraz aplikuje konfigurację zdefiniowaną przez użytkownika.

```
public static IServiceCollection UseAJT(this IServiceCollection services,
                                       AJTOptions options,
                                       Func<IAJTConfigurator, IAJTConfigurator> configure)
{
    // Rejestracja warstwy danych
    services.AddDbContext<AJTDbContext>();
    services.AddScoped<IUserRepo, UserRepo>();
    services.AddScoped<IRoleRepo, RoleRepo>();
    services.AddScoped<IUserRoleRepo, UserRoleRepo>();
    services.AddScoped<IRefreshTokenRepo, RefreshTokenRepo>();

    // Konfiguracja opcji i serwisów pomocniczych
    var configurator = (AJTConfigurator)configure(new AJTConfigurator());
    configurator.ApplyConfiguration(services, options);

    services.AddSingleton(MOptions.Create(options));

    // Rejestracja serwisów logiki biznesowej
    services.AddSingleton<IHashingService, HashingService>();
    services.AddSingleton<ILoginService, LoginService>();
    services.AddSingleton<IRoleService, RoleService>();
    services.AddSingleton<ITokenDataService, TokenDataService>();

    return services;
}
```

9 Praktyczne zastosowanie w warstwie API

Weryfikacja poprawności działania zaprojektowanego systemu autoryzacji odbywa się poprzez jego integrację z warstwą kontrolerów aplikacji ASP.NET Core. Poniższy przykład ilustruje sposób wykorzystania atrybutów `[RequireAuthentication]` oraz `[AllowRole]` do zabezpieczania punktów końcowych, a także użycie serwisu `IRoleService` do dynamicznego zarządzania uprawnieniami.

9.1 Implementacja kontrolera testowego

Kontroler `ApiController` eksponuje metody o różnym poziomie dostępu. Warto zwrócić uwagę na:

- Metody `GrantRole` i `RemoveRole` – pozwalają na dynamiczne modyfikowanie uprawnień użytkownika w czasie rzeczywistym, wykorzystując wstrzyknięty serwis `IRoleService`.
- Metodę `Data` – demonstruje dostęp do danych wstrzykniętych do tokenu (np. "siema") poprzez słownik `HttpContext.Items`.
- Metody z atrybutami `AllowRole` – obrazują deklaracyjny sposób ograniczania dostępu dla konkretnych ról (np. "admin", "role1").

```
using AJT.Attributes;
using AJT.Contracts;
using Microsoft.AspNetCore.Mvc;

namespace API.Controllers
```

```

{
    [ApiController]
    [Route("api")]
    public class ApiController : Controller
    {
        private readonly IRoleService _roleService;

        public ApiController(IRoleService roleService)
        {
            _roleService = roleService;
        }

        [HttpPost("role/grant")]
        public async Task<IActionResult> GrantRole([FromBody] GrantRequest request)
        {
            await _roleService.AddUserRole(request.UserId, request.Role);
            return Ok();
        }

        [HttpPost("role/remove")]
        public async Task<IActionResult> RemoveRole([FromBody] GrantRequest request)
        {
            await _roleService.RemoveUserRole(request.UserId, request.Role);
            return Ok();
        }

        [RequireAuthentication]
        [HttpGet("data")]
        public IActionResult Data()
        {
            var data = HttpContext.Items["AJT"] as string;
            return Ok($"stored token data: {data}");
        }

        [AllowRole("admin")]
        [HttpGet("endpoint")]
        public IActionResult Admin() => Ok("admin");

        [AllowRole("admin", "role1")]
        [HttpGet("admin1")]
        public IActionResult Admin1() => Ok("admin and role1");

        [AllowRole("role1")]
        [HttpGet("role1")]
        public IActionResult Role1() => Ok("role1");

        [AllowRole("role2")]
        [HttpGet("role2")]
        public IActionResult Role2() => Ok("role2");

        public sealed class GrantRequest
        {
            public Guid UserId { get; set; }
            public string Role { get; set; }
        }
    }
}

```

9.2 Inicjalizacja i konfiguracja środowiska

Ostatnim elementem układanki jest plik `Program.cs`, w którym następuje spięcie wszystkich komponentów. Wywołanie metody `UseAJT` konfiguruje parametry kryptograficzne (klucz sekretny), czasy życia tokenów oraz definiuje pulę dostępnych ról (w tym przypadku generowanych dynamicznie w celach testowych).

Widoczna jest również konfiguracja `RoleBootstrapper` oraz włączenie automatycznych migracji bazy danych, co czyni bibliotekę samowystarczalną ("plug-and-play") przy pierwszym uruchomieniu w nowym środowisku.

```
using AJT;
using AJT.Contracts;
using API.Services;

namespace API
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            builder.Services.AddControllers();

            // Rejestracja i konfiguracja biblioteki AJT
            builder.Services.UseAJT(
                new AJT.Options.AJTOptions
                {
                    ConnectionString = builder.Configuration.GetConnectionString("AJT"),
                    Secret = "super_sekretny_klucz",
                    TokenExpirationTime = TimeSpan.FromMinutes(10),
                    RefreshTokenExpirationTime = TimeSpan.FromDays(7),
                    Roles = GenerateRoles()
                },
                config => config.UseRoleBootstrapper()
                // .UsePasswordHashing<PasswordHashingService>()
                // .AutomaticallyDetectRoles()
                .MigrateDatabase()
                .AddDataToToken(AddInfo)
            );

            var app = builder.Build();

            app.UseHttpsRedirection();
            app.UseAuthorization(); // Middleware autoryzacji
            app.MapControllers();

            app.Run();
        }

        // Delegat wstrzykujący dodatkowe dane do tokenu
        static async Task<object> AddInfo(Guid userId, IServiceProvider services)
        {
            return "siema";
        }

        // Generator ról testowych
        static List<string> GenerateRoles()
        {
            var roles = new List<string> { "admin" };
            for (int i = 0; i < 50; i++)
                roles.Add($"role{i}");
            return roles;
        }
    }
}
```



```
}  
    }  
}
```

10 Podsumowanie

Przedstawiony projekt systemu autoryzacji opartego o funkcjonalność tokenów stanowi kompletną, modułową bibliotekę dla środowiska .NET. Rozwiązanie to adresuje kluczowe wyzwania współczesnych aplikacji webowych, oferując:

- **Wydaźność:** Zastosowanie bitowego mapowania ról (ciąg Hex) oraz lekkiej struktury tokenu (bez nagłówka JSON) znacząco redukuje narzut danych przesyłanych w sieci oraz przyspiesza proces weryfikacji uprawnień.
- **Bezpieczeństwo:** Wykorzystanie algorytmu HMAC-SHA256 do podpisywania tokenów oraz mechanizmu *Refresh Token* zapewnia integralność danych i bezpieczne zarządzanie sesją użytkownika.
- **Elastyczność:** Architektura oparta na wstrzykiwaniu zależności oraz interfejsach pozwala na łatwą wymianę komponentów (np. bazy danych, algorytmu haszowania haseł) oraz rozszerzanie funkcjonalności (wstrzykiwanie własnych danych do tokenu).
- **Automatyzację:** Mechanizmy takie jak *RoleBootstrapper* oraz automatyczne migracje upraszczają proces wdrażania i utrzymania spójności systemu.

Projekt spełnia założenia pracy inżynierskiej, dostarczając gotowe do użycia narzędzie, które może stanowić alternatywę dla bardziej złożonych systemów tożsamości w aplikacjach o specyficznych wymaganiach wydajnościowych.